

From Pixels to Patches

Pooling Strategies for Earth Embeddings

Isaac Corley¹ Caleb Robinson² Inbal Becker-Reshef² Juan M. Lavista Ferres²

ICLR 2026 | ML4RS Workshop

¹Wherobots

²Microsoft AI for Good Research Lab

Motivation

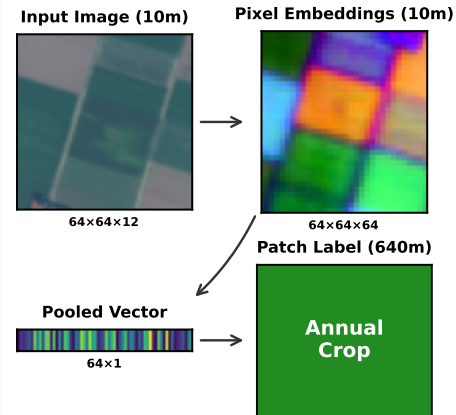
The resolution mismatch problem

Geospatial foundation models produce **pixel-level** embeddings (64–512 dims per pixel).

But downstream labels live at **patch / region** scale — land cover patches, crop fields, building polygons.

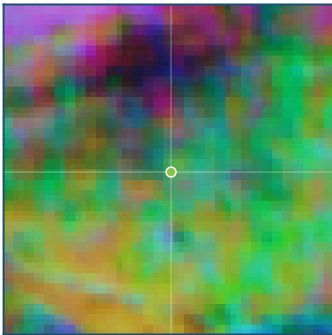
So we must pool. The question is *how*.

The default — mean pooling — drops accuracy by >10% under geographic shift.



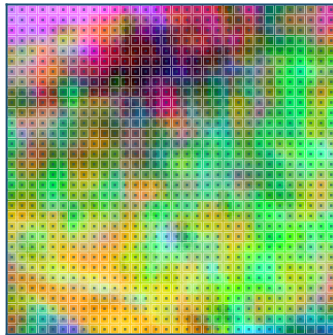
Patch vs. pixel embeddings

PATCH Embedding



One vector per patch

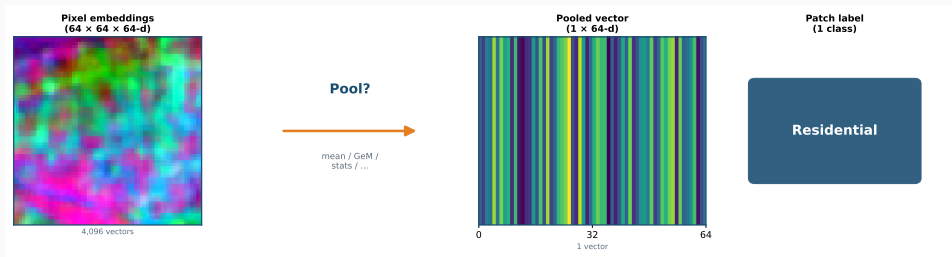
PIXEL Embedding



One vector per pixel

Pixel-level models produce a vector *per pixel* — thousands of embeddings per patch that must be aggregated.

The pooling pipeline



PCA pseudo-RGB of actual AlphaEarth embeddings (64-d). Pooling compresses 4,096 vectors into one — the method matters.

Pooling matters in other domains

Image Retrieval

GeM pooling consistently
outperforms mean pooling

Radenović et al., 2018

Audio / Video

Stats pooling (mean+std)
summarizes
variable-length
sequences effectively

Miech et al., 2017

Object-Based IA

Bag of Visual Words
uses region histograms
for classification

Csurka et al., 2004

Yet pooling is **largely unexplored** for earth embeddings.

EuroSAT-Embed

EuroSAT-Embed: a pooling research benchmark

81,000 pixel-embedding GeoTIFFs from three foundation models, aligned to EuroSAT (27k patches, 10 land-cover classes).

AlphaEarth

64-d, int8, 3.3 GB

OlmoEarth-Nano

128-d, float32, 40 GB

Tessera

512-d, float32, 164 GB

Reproducible pooling research **without re-running** expensive GFM inference.

Evaluated on **random** (i.i.d.) and **spatial** (geographically disjoint) splits.

CC-BY-4.0 <https://hf.co/datasets/isaaccorley/eurosat-embed>

Pooling Methods

First-order pooling

Given $X \in \mathbb{R}^{H \times W \times D}$ with $N = HW$ pixels:

Mean (baseline)

$$\mu = \frac{1}{N} \sum_i x_i$$

Max — per-channel maximum

Generalized Mean (GeM)

$$\text{GeM}_p = \left(\frac{1}{N} \sum_i x_i^p \right)^{1/p}$$

Center-weighted mean — Gaussian kernel

All preserve original dimensionality ($1\times$).

Hypothesis: class signal lives in the *distribution*, not just the mean.

Distributional (training-free):

- Mean+Std, Mean+Max (2×)
- Median+IQR (2×)
- Stats [min/max/ μ/σ] (4×)
- Percentiles [10..90] (5×)
- Covariance upper-tri

Parametric (need training data):

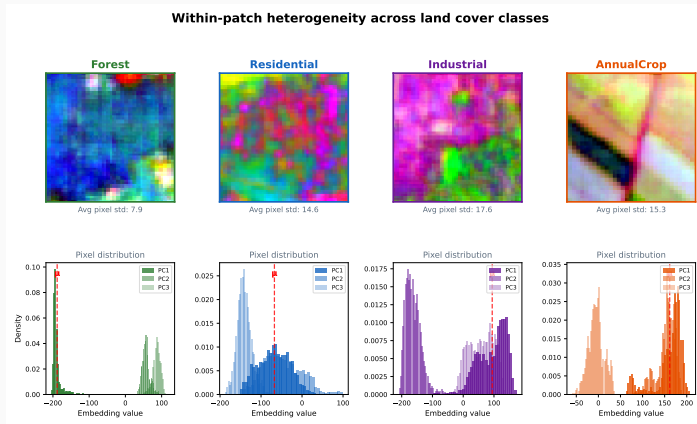
- PCA — project to lower dim
- BoVW — k -means histogram

E.g., a residential patch and an industrial area can share a mean but differ wildly in variance.

Within-patch heterogeneity across classes

Homogeneous classes
cluster tightly;
heterogeneous ones show
broad distributions.

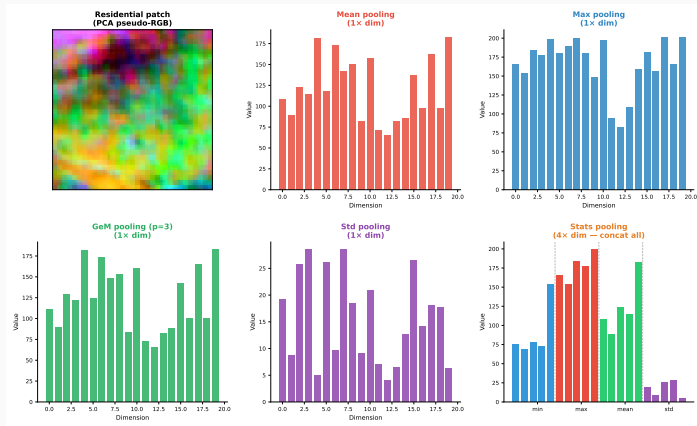
Mean pooling (μ , red)
misses this signal.



What different pooling methods capture

Same patch, different pooling outputs.

Stats (4× dim) captures the full distribution; mean (1×) collapses it.



Setup

Evaluation protocol



= 156 experiments

kNN ($k=5$, cosine distance)

Probes geometric / similarity structure

Linear (logistic regression, L-BFGS)

Probes linear separability

Results

Distributional statistics improve generalization

Linear probe on **spatial** splits:

Mean (baseline) 76–84%

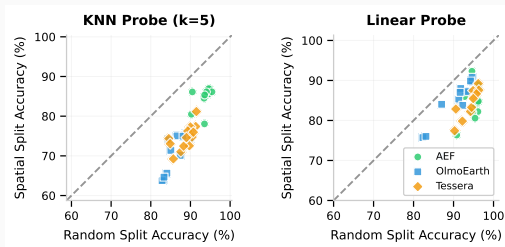
Mean+Max 87–89%

Stats 88–91%

+7–10% gain over mean pooling.

On random splits, differences shrink to 3–5% (spatial leakage flatters everyone).

The spatial split is the honest evaluation.



Closer to diagonal = smaller gap

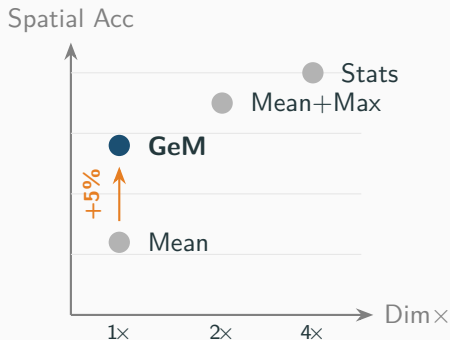
GeM: zero-cost upgrade

Generalized Mean ($p=3$):

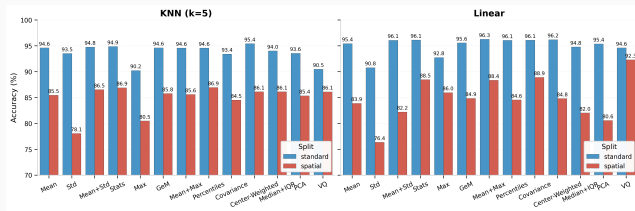
$$\text{GeM} = \left(\frac{1}{N} \sum_i x_i^3 \right)^{1/3}$$

- +5% spatial accuracy over mean
- Same dimensionality (1x)
- No training, no storage overhead
- Consistent across all three encoders

Softly upweights high-activation pixels without the harsh cutoff of max pooling.



Accuracy–dimensionality tradeoff



No overhead (1×)

GeM (+5%)

Best tradeoff (2×)

Mean+Max (+6–8%)

Peak accuracy (4×)

Stats (+7–10%)

Pooling interacts with the encoder

AlphaEarth (64-d)

BoVW hits 92.3% (linear) but fails on other encoders.

Compact embeddings are naturally more robust to shift.

OlmoEarth (128-d)

Std alone boosts kNN by +11% over mean.

Second-order stats help similarity search more than linear probes.

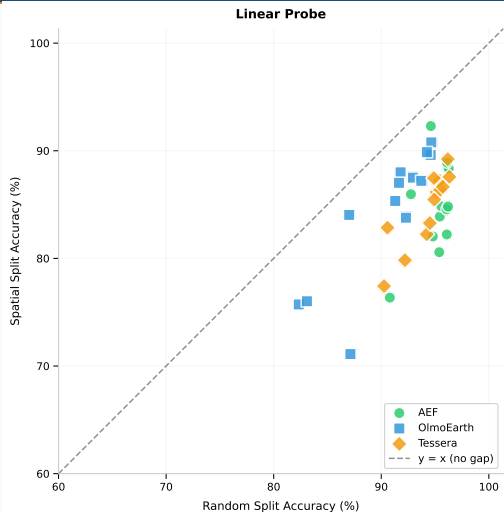
Tessera (512-d)

Covariance pooling excels at kNN.

Largest mean-pooling gap (12%). High-dim benefits most from stats.

Higher embedding dimension $\implies$ more benefit from distributional statistics.

Richer statistics close the generalization gap



Accuracy drop (random $\rightarrow$ spatial):

Mean **10%** drop

Mean+Max 6.4% drop

Stats 6.2% drop

Max 5.8% drop

Mean pooling discards intra-patch heterogeneity that matters most under geographic shift.

Takeaways

Recommendations

Scenario	Method	Gain	Dim
Drop-in default	GeM ($p=3$)	+5%	1×
Best accuracy	Stats [min/max/ μ/σ]	+7–10%	4×
Balanced	Mean+Max	+6–8%	2×
kNN retrieval	Covariance / Percentiles	+3–7%	varies

All recommended methods are **training-free** and consistent across encoders.

Start with GeM. Upgrade to Stats if you can afford 4× the storage.

Why does this work?

The intuition

Mean pooling collapses a patch into a single “average pixel.”

A residential neighborhood and an industrial park can share a similar mean but differ in variance — mixed rooftops, roads, vegetation vs. uniform warehouses.

Higher-order statistics preserve that heterogeneity.

Broader implications

Extends GeM results from image retrieval to the geospatial domain.

Foundation models encode fine-grained semantic structure worth preserving.

Aggregation should be a design decision, not an afterthought.

Limitations

- Single benchmark (EuroSAT)
- Three embedding sources
- Fixed hyperparameters

Next steps

- BigEarthNet, fMoW
- Learned / attention pooling
- Temporal aggregation

Open resources

Dataset: <https://hf.co/datasets/isaaccorley/eurosat-embed>

Code: <https://github.com/isaaccorley/geopool>

Questions?



Code



Dataset